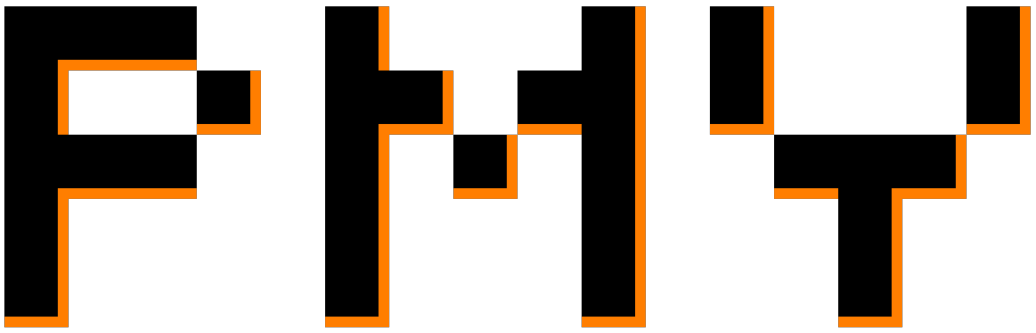


MALİK ENES
ŞAFAK

PMY-WEB.GITHUB.IO

Mesleki Matematik

Dijital Sinyal 0.1

İçindekiler

Ayrık Zamanlı Sinyaller (Discrete Time Signals) 3

Sinyal 3

Örnekleme 4

Ayrık Zamanlı Sistemler (Discrete Time Systems) 12

Sistem 12

Lineer Sistemler (Linear Systems) 13

Zamanda Değişmez Sistemler (Time-Invariant System) 13

Nedensellik (Causality) 15

Stabilite (Stability) 16

LTI (Linear time-invariant) Sistemler 17

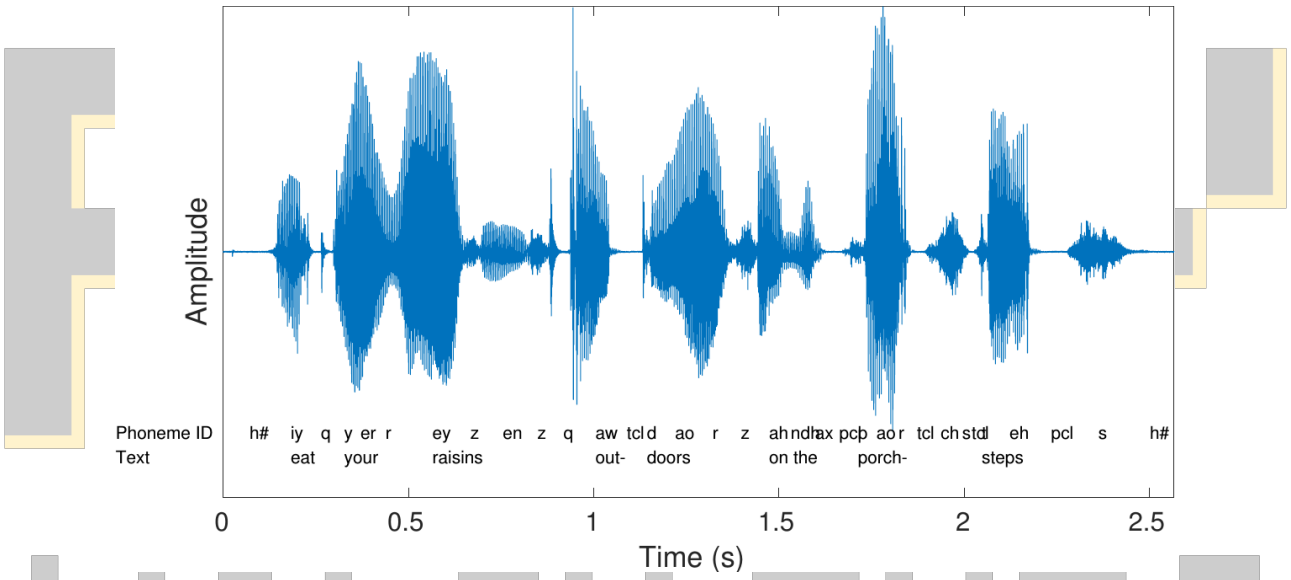
MALİK ENES ŞAFAK

PMY-WEB.GITHUB.IO

Ayrık Zamanlı Sinyaller (Discrete Time Signals)

Sinyal

Sinyal, en basit haliyle, bir bilgiyi taşıyan herhangi bir şeydir. Sinyaller bir sistemin o anki durumunu veya davranış şeklini anlatabilir. Örneğin, konuşma sinyali zamanın bir fonksiyonu olarak, resim sinyali ise iki boyutlu bir parlaklık fonksiyonu olarak ifade edilebilir.



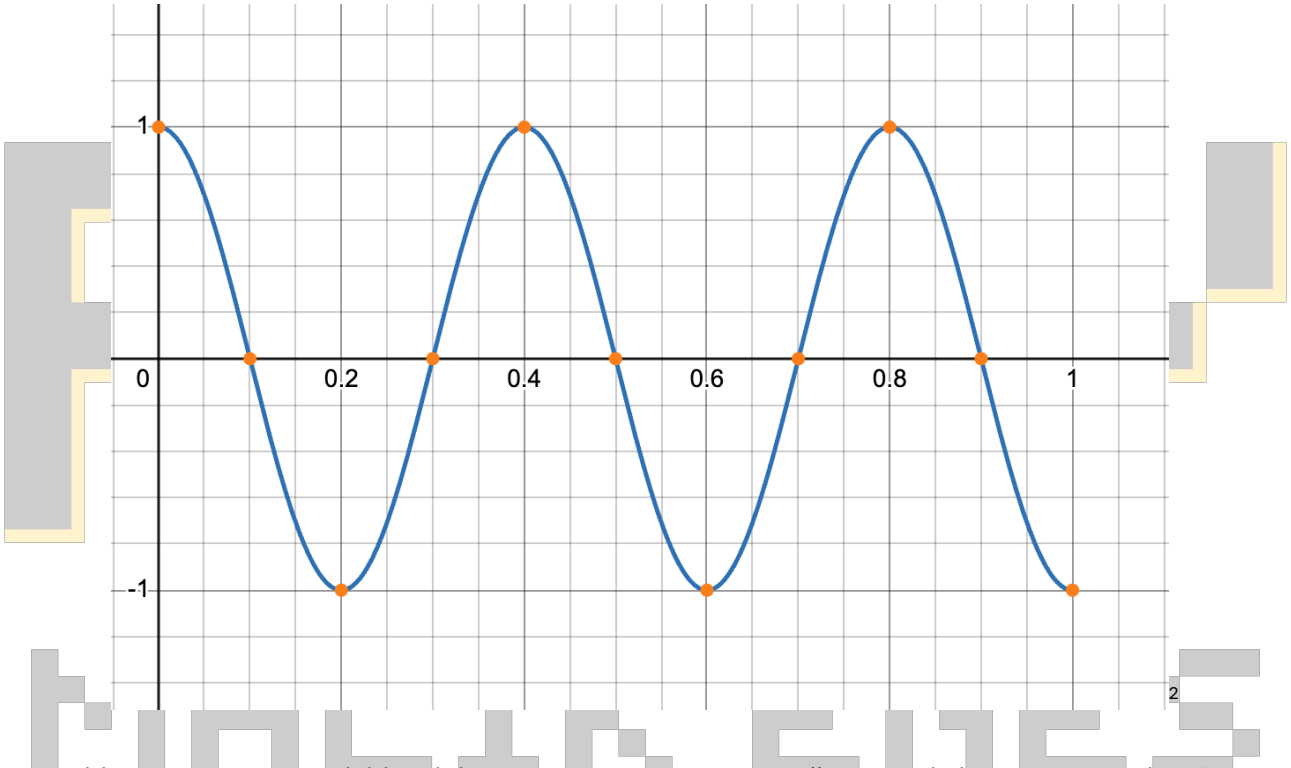


Normalde sürekli zamanlı (continuous time, yaygın kullandığımız şekliyle analog) olan bu fonksiyonların dijital olarak iletilebilmesi için örneklenmesi gereklidir.

Örnekleme

Örnekleme, sürekli zamanlı olan bir fonksiyonun ayrık zamanlı bir diziye dönüştürülmesi işlemi olarak tanımlanabilir. Dolayısıyla ayrık zamanlı sinyaller matematiksel olarak sayılar dizisi ile temsil edilir.

$$y[n] = \sin(2\pi \mathrm{fnT})$$ [1]



[2]

Yukarıdaki ifadede **n** sayısı dizi içerisinde kaçıncı eleman olduğunu belirtirken **T** sayısı ise örnekleme periyodunu belirler. **Örnekleme periyodu** sürekli zamanlı sinyal içerisinden kaç saniyede bir örnek alınacağını gösterir. Örnekleme periyodu

$$f_s = \frac{1}{T}$$

formülü ile örnekleme frekansına çevrilebilir. **Örnekleme frekansı ya da örnekleme oranı** ise 1 saniyede kaç adet örnek alınacağını gösterir. Dolayısıyla yukarıdaki formül

$$y[n] = \sin\left(2\pi f \frac{n}{f_s}\right)$$

olarak da yazılabilir.

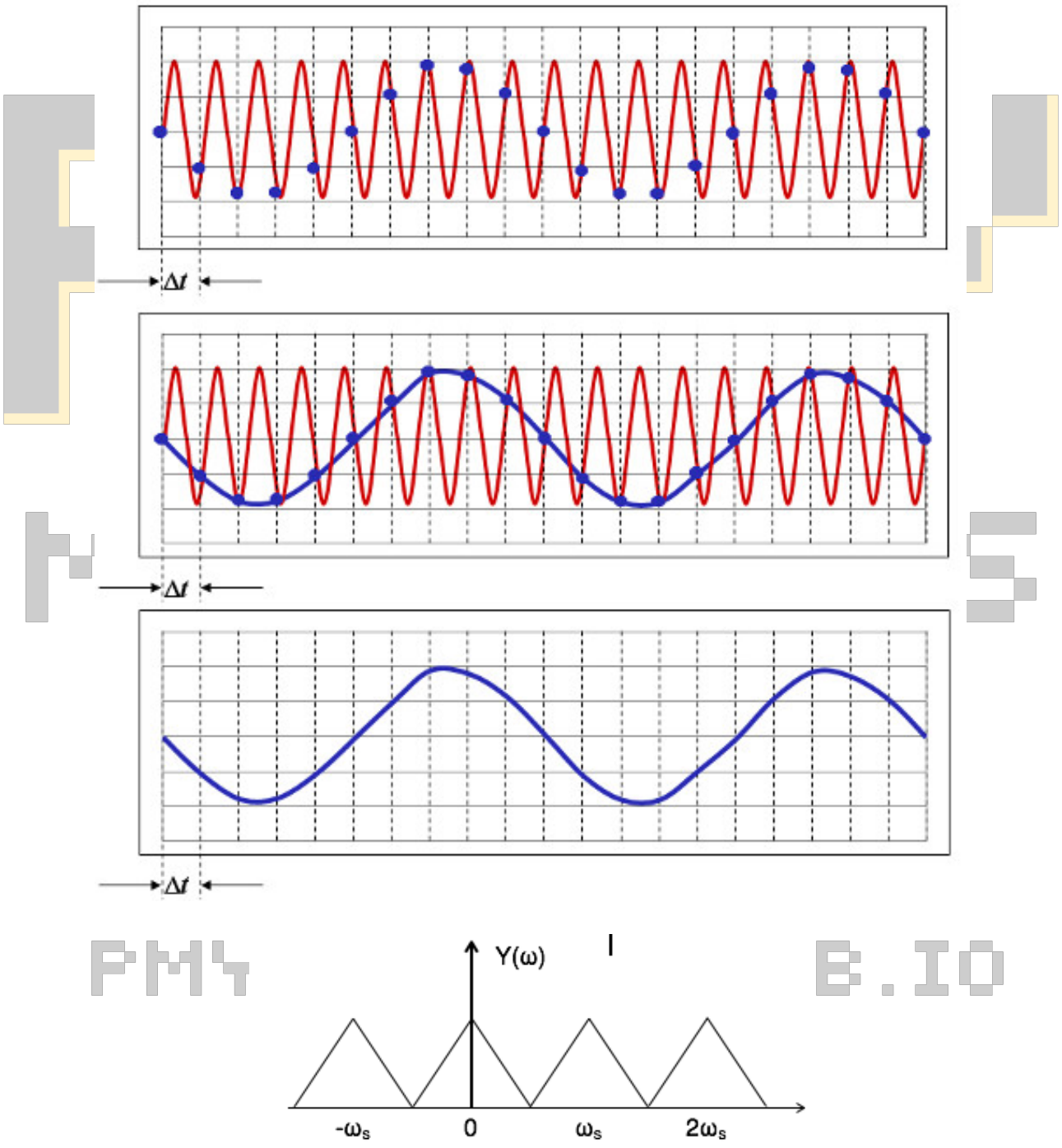
---

[1] https://www.desmos.com/calculator/jjrpgz4eeo

[2] Buradaki görsele bakarak y dizisinin $[1, 0, -1, 0, 1, 0, -1, 0, 1, 0, -1]$ olduğunu söyleyebiliriz (diziler genellikle köşeli parantez içerisinde gösterilir). Her bir örneğin arasındaki mesafeye bakarak ise örnekleme periyodunun 0.1 saniye, dolayısıyla 10 Hz olduğu sonucuna varabiliriz. Peki saniyede 10 adet örnek alıyorsak görselde neden 11 adet örnek var?

Bir sinyalin herhangi bir bozulmaya uğramadan örneklenebilmesi için içerisinde bulunan en yüksek frekansın en az iki katı kadar örnek alınması gerekir[3]. Bu sebeple örnekleme frekansının yarısı olan frekansa (yani örneklenebilecek maksimum frekansa) **Nyquist frekansı** adı da verilmektedir.

Eğer Nyquist frekansından daha yüksek frekanslı bir sinyal örneklenirse bu durumda **katlanma (aliasing)** ortaya çıkar.



[3] Bu teorem Nyquist-Shannon teoremi olarak geçer. Nyquist ve Shannon bu fenomeni birbirlerinden haberdar olmadan keşfetmişlerdir. Shannon'un teorem hakkında açıklaması için https://www.essrl.wustl.edu/~jao/itrg/shannon.pdf (19. Bölüm, Band Limited Ensembles of Functions)

Yukarıdaki grafikte x ekseni sisteme verilen frekansı, y ekseni ise sistemin örnekleyeceği frekansı göstermektedir. ω_s ise örnekleme frekansıdır[4].

Sisteme giren f frekansına karşılık sistemin örnekleyeceği frekans

$$y(f) = \begin{cases} mod(f, f_s), & mod(f, f_s) < \frac{f_s}{2} \\ \frac{f_s}{2} - mod(f, \frac{f_s}{2}), & mod(f, f_s) \geq \frac{f_s}{2} \end{cases}$$

formülü ile hesaplanabilir[5].

Matematiksel olarak aliasing'i şöyle kanıtlayabiliriz, örneğin örnekleme frekansının 1.25 katında bir sinüs dalgasını örneklemek istediğinizi düşünün. Bu frekansı

$$1.25f_s = 1f_s + 0.25f_s$$

olarak yazabiliriz. Bu durumda örnekleyeceğimiz sinyali

$$y[n] = \sin\left(2\pi 0.25 f \frac{n}{f_s} + 2\pi f_s \frac{n}{f_s}\right)$$

olarak yazabiliriz. Sinüs fonksiyonu içerisinde toplamanın sağındaki kısım, n her zaman tam sayı olması gerektiği için, birim çember etrafında 2π dönmeye sebep olacaktır. Eğer birim çember etrafında 2π kadar dönerseniz, birim çemberin çevresi kadar döneceğiniz için, aynı yere geri dönmüş olursunuz. Dolayısıyla toplamanın sağındaki kısım örneklenecek sinyal üzerinde etkisini kaybedecek, sadece toplamanın solundaki kısım örneklenmiş sinyale dahil olacaktır.

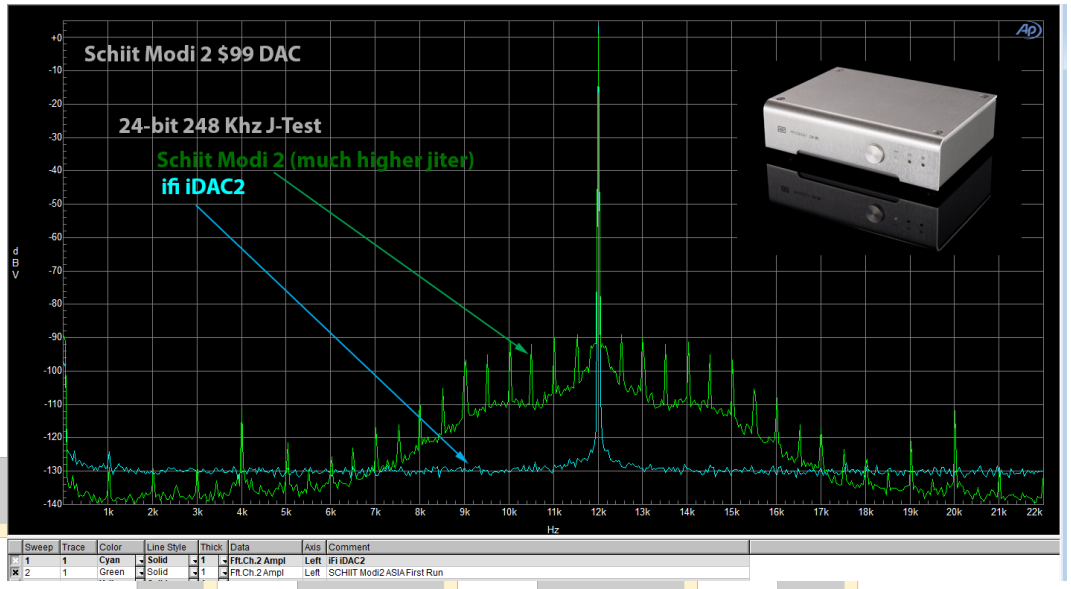
Örnekleme esnasında alınan örneğin tam örnekleme periyodunda alınması önemlidir. Yani 0.1 saniye örnekleme periyoduna sahip bir sistemde 0.1 yerine 0.11'de örnek alırsanız elde edeceğiniz sonuç, siz daha sonradan bu örneği 0.1 saniyede alınmış gibi değerlendireceğiniz için, bozulmuş olur[6]. Bu probleme **faz bozulması, zaman sapması ya da jitter** adı verilir.

FMY-WEB.GITHUB.IO

[4] https://www.desmos.com/calculator/onk7uwcfb8

[5] Bunun yerine yukarıdaki grafiği aklınıza getirerek hesaplamak çok daha kolay olacaktır.

[6] Burada aklınıza "Öyle değerlendirmesek olmaz mı?" sorusu gelebilir. Maalesef olmaz. Bunun birkaç farklı sebebi vardır. İlk olarak örneklemenin sabit bir periyotta yapılıyor olması örneğin alındığı zamanı kaydetme ihtiyacını ortadan kaldırır. İkinci olarak sistem örnek almak için aynı miktarda gecikiyorsa bu zaten çok büyük bir sorun değildir (ses olarak düşünürsek geri oynattığınızda sesi orijinalinden biraz daha tiz ya da pes duyarsınız). Fakat böyle bir gecikme genellikle farklı miktarlarda gerçekleşir (çoğunlukla örnekleme donanımından kaynaklanan bir sorundur, yani ses kartınızdan). Bu da örneklediğiniz sinyalin içerisinde var olmayan frekansların oluşmasına sebep olur. Son olarak insan algısı, özellikle çok düşük frekanslı harekette (yani görüntüde ya da ışıkta) periyodikliği bozulan şeylere alışkın değildir, dolayısıyla bu bozulmadan rahatsız olur. Sesteki etkisini duymak için https://www.youtube.com/watch?v=EPhEOWNdBgc, görüntüdeki etkisini görmek için https://www.youtube.com/watch?v=5SSU-s0AUH0



Jitter, ses kartlarında, çoğunlukla J-Test adı verilen yöntem ile ölçülür. Bu yöntem örnekleme frekansının dörtte birinde ya da Nyquist frekansının yarısında bir kare dalganın ses kartının girişine verilerek frekans ekseninde analiz edilmesiyle gerçekleştirilir. Kare dalga Nyquist frekansının yarısında olduğu ve kare dalganın bir sonraki harmoniği Nyquist frekansının 1.5 katında olacağı için frekans spektrumunun tek bir sinüs dalgası göstermesi gerekir[7] (yukarıdaki resimde mavi çizgi buna bir örnektir). Eğer başka frekanslar da ortaya çıkıyor ise kullanılan donanımda jitter problemi var demektir.

Bir sinyali örneklerken, belirli bir örnekleme periyodunda, sinyalin genliği ile ilgileniriz. Bu genlik verisini bir yerden başka bir yere iletebilir, kaydedebilir veya üzerinde işlem yapabiliriz. Analog bir sinyali kaydederken çoğunlukla bir çeşit yuvarlama yapmamız gerekir çünkü sonsuza kadar giden bir sayıyı[8], yeteri kadar yerimiz olmadığı için, kaydedemeyiz.

Ses sinyalini örneklerken, ses kartımız, genlik değeri olarak voltajı ölçer. Çünkü biz ses sinyalini, standart olarak, voltaj farkı üzerinden iletiriz[9]. Ve yine standart olarak bu voltajın -1 Volt ile +1 Volt aralığında olacağını kabul ederiz. Örneğin bir mikrofonu ses kartınıza bağladığınızda mikrofonunuz

[7] Çünkü ses kartınız girişine gelen sinyalin, çalıştığı örnekleme oranına göre, Nyquist frekansının üzerinde kalan frekanslarını kesmesi gerekir. Bu durumda kare dalganın bütün harmonikleri filtre tarafından kesileceği için sadece temel harmonik kalır, yani tek bir sinüs dalgası görmeniz gerekir. Eğer ses kartınız bu filtrelemeyi yapmıyor olsaydı sürekli aliasing problemiyle karşılaşırdınız. Çünkü sadece elektronik ya da dijital olarak üretilen sinyaller değil, mikrofon tarafından alınan sinyaller de 20kHz'in üzerinde frekanslar içerir.

[8] Burada örnek olarak π sayısını düşünebilirsiniz. π sayısı sonsuza kadar uzar. Bu metni okuduğunuz ekrana baktığınızda bile yuvarlak ya da kenarları yuvarlatılmış bir sürü şey görebilirsiniz. Bunların hepsinin çizilebilmesi için bir şekilde π sayısına ihtiyacımız var. Eğer bunları hesaplamak için π sayısının tamamını bilgisayarımızda (ya da telefonumuzda) tutacak olsaydık, sizce ne kadar alana ihtiyacımız olurdu? Teorik olarak sonsuz, fakat biz π sayısının tamamını henüz bilmiyoruz. 2 Nisan 2025 tarihi itibariyle "sadece 300,000,000,000,000" basamağını biliyoruz. Her bir basamağın 1 byte yer kapladığını düşünsek yaklaşık 273 terabyte alana ihtiyacımız olurdu. Dolayısıyla bir noktada "bu kadar basamak bize yeter" dememiz gerekir.

[9] Başka şekillerde de iletebilirdik fakat kabul ettiğimiz standart bu şekilde. Ayrıca, voltaj farkı ve voltaj aynı anlama gelmektedir. Voltaj her zaman iki farklı değer arasındaki farkı bize söyler. Günümüzde çoğunlukla bu fark toprak (0 Volt) ile sinyal arasındaki fark olduğu için voltaj farkı demek yerine sinyalin voltajı demek de uygun bir kullanım şeklidir.

akustik enerjiye karşılık olarak voltaj farkı üretir. Ses kartınız ise bu voltaj farkını ölçerek sayılara dönüştürür. Bu dönüşüm esnasında ses kartınız voltajı bir şekilde yuvarlamak zorundadır çünkü ne kadar hassas ölçerseniz ölçün voltajın bir noktada bittiğini göremezsiniz[10].

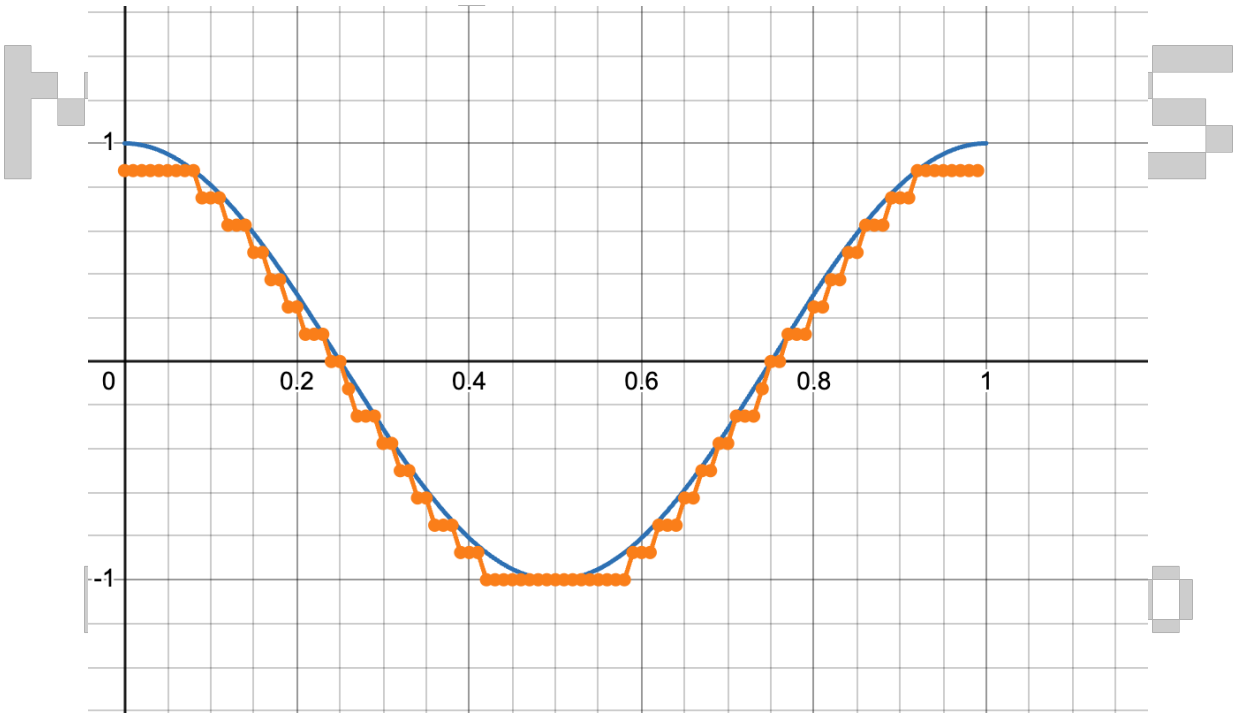
Bilgisayar içerisinde ayrık zamanlı sinyallerin genlik bilgisini saklarken genellikle **bit derinliğinden (bit depth)** bahsederiz. Bit derinliği her bir örneğin kaç bit (ikilik basamak) kullanılarak saklanacağını belirler. Örneğin her bir örnek 4-bit olarak saklanıyorsa analog olarak ölçeceğimiz voltajı temsil edebileceğimiz

$$2^4 = 16$$

farklı değerimiz var demektir. Eğer -1 ile +1 Volt arasında kalan voltajı örnekliyorsak toplam örnekleyeceğimiz aralık 2 Volt olacağı için örneğimizi

$$\frac{2\ Volt}{16} = 0.125\ Volt$$

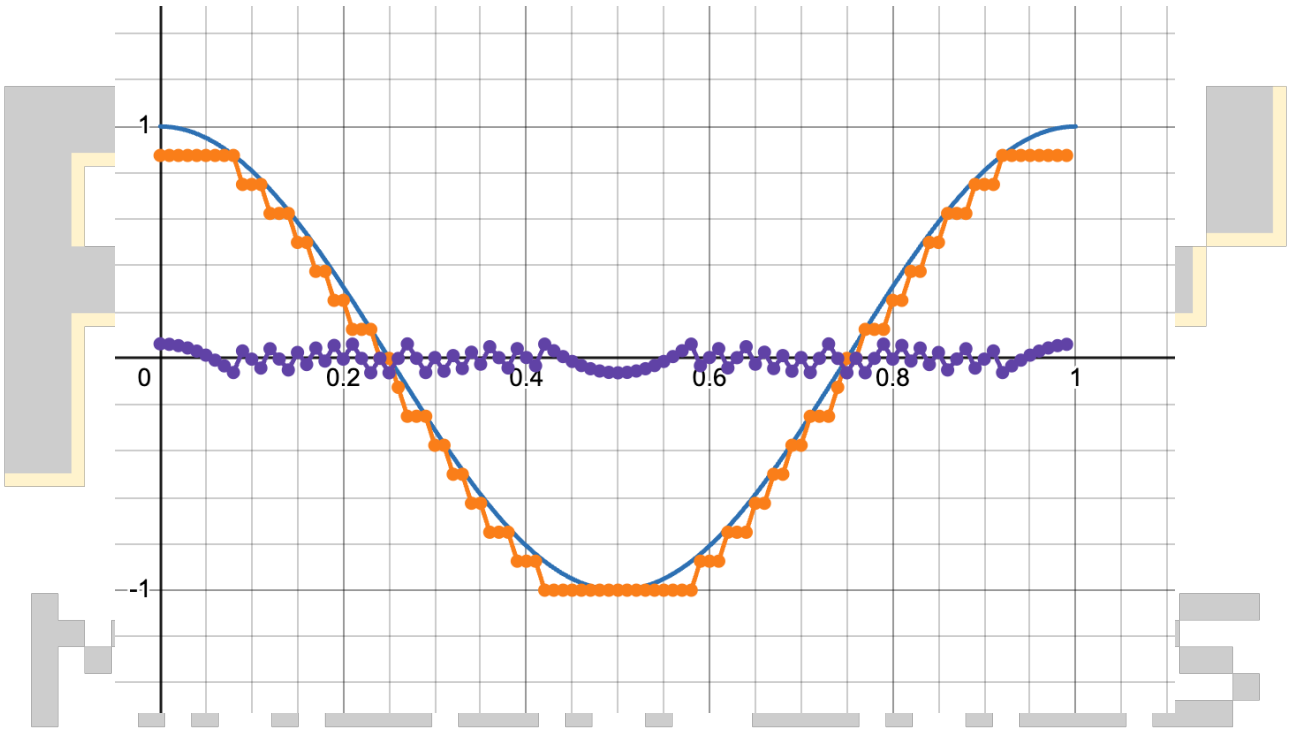
katlarındaki sayılara yuvarlamamız gerekir. Bu yuvarlama işlemine **kuantizasyon (quantization) ya da yuvarlama (rounding)** adı verilir[11].



[10] Bu demek değildir ki analog sinyaller ya da ekipmanlar sonsuz çözünürlüğe sahiptir. Bir noktadan sonra ölçeceğiniz şey mutlaka gürültü olacaktır. Analog ekipmanların sonsuz çözünürlüğe sahip olduğuyla ilgili genel bir yanlış kanı bulunmaktadır. Bir sinyali, üzerinde değişiklik yapmadan, sonsuz çözünürlükte saklayacak ya da iletecek hiçbir sistem yoktur.

[11] https://www.desmos.com/calculator/peqydyheg8

Kuantizasyon işleminden sonra ortaya çıkan örnekler örneklenen sinyalin birebir temsilidir. Bit derinliğinin ne kadar az ya da fazla olduğu örneklenmiş sinyalin çözünürlüğüne etkisi yoktur. Yani, 1kHz olan bir sinüs dalgası örnekleniyorsa, bit derinliği ne olursa olsun 1kHz sinüs dalgası elde edilecektir. Bit derinliği sadece gürültüyü etkiler ve kuantizasyon işlemi sonucu ortaya çıkan gürültüye **kuantizasyon gürültüsü (quantization noise)** adı verilir. Bu gürültünün kaynağı orijinal sinyal ile yuvarlanmış versiyonu arasındaki farktır[12].



Bu gürültünün seviyesini

$$FS_{qn} = \frac{1}{2^{d_b}}$$

formülüyle hesaplayabiliriz. FS_{qn}, kuantizasyon gürültüsünün FS değerini, d_b ise bit derinliğini göstermektedir. Örneğin 4-bit ile örneklenmiş bir sinyal içerisindeki gürültünün seviyesinin

$$FS_{qn} = \frac{1}{2^4} = \frac{1}{16} = 0.0625FS \approx -24dBFS$$

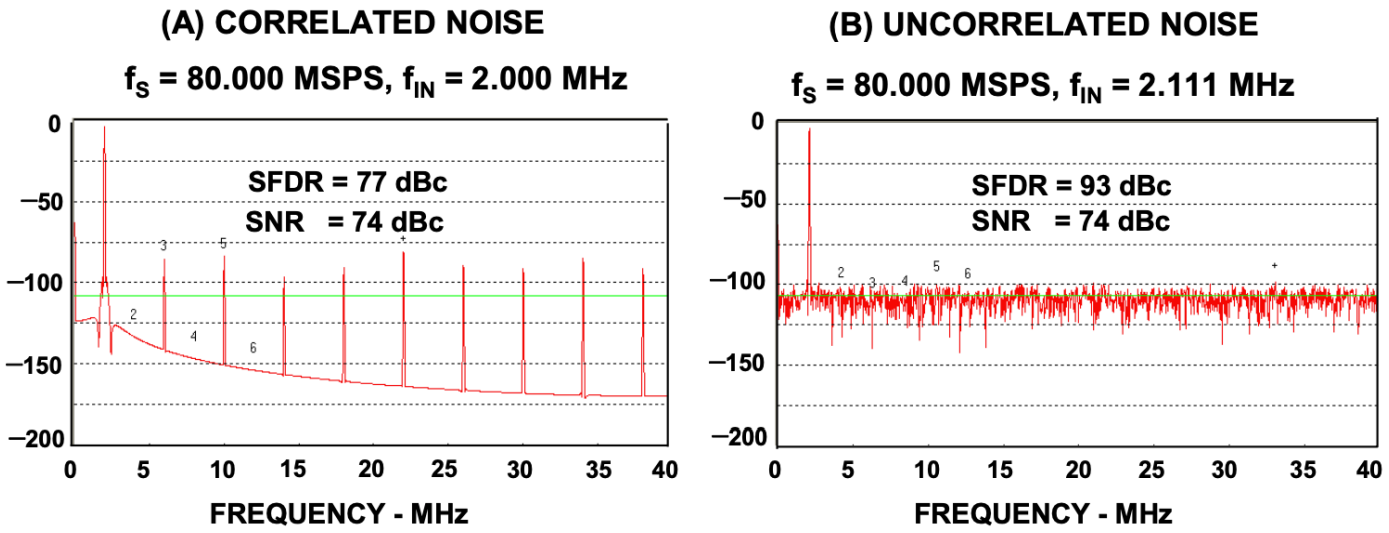
olacağını hesaplayabiliriz[13].

[12] https://www.desmos.com/calculator/hif29eloin

[13] Bir sinyal içerisindeki kuantizasyon gürültüsünü anlatmak için kullanılan bir diğer kavram SQNR'dır (Signal to Quantization Noise Ratio, Sinyal ile Kuantizasyon Gürültüsünün Oranı). Yukarıdaki örneğe baktığımızda eğer sinyalimiz 1 FS ise 4-bit için SQNR'ın 0.0625 olduğunu söyleyebiliriz ($\frac{1}{0.0625} = 0.0625$). Fakat sinyalimiz 0.0625 FS ise SQNR'ımız 1 olacaktır ($\frac{0.0625}{0.0625} = 1$). Bu durumda diyebiliriz ki, SQNR değerimiz 1 veya daha büyük bir sayı ise sinyalimizin seviyesi kuantizasyon gürültüsünün altında kalacak ve dolayısıyla duyulmayacaktır.

Yukarıda görüleceği üzere bu gürültü örneklenen sinyalin içeriğine çok bağlıdır ve periyodik hareketler oluşturabilir. Bu periyodik hareketler sinyal içerisinde var olmayan frekansların ortaya çıkmasına sebep olabilir. Bu nedenle kuantizasyon esnasında gürültünün belirli bir frekansta toplanmasının önlenmesi ve beyaz gürültü gibi duyulması için **dithering** yapılması gerekir.

Dithering işlemi kuantizasyon gürültüsünün enerjisinin belirli frekanslarda toplanmasını önleyerek bütün frekans eksenine yayar. Gürültünün toplam enerjisi sabit kalsa da bütün frekanslara yayıldığı için çok daha az rahatsız edici duyulur[14]. Kısacası, dithering, kuantizasyon işlemi sonucunda oluşan gürültünün örneklenen sinyal ile bir ilişkisinin kalmamasını ve tamamen beyaz gürültü gibi duyulmasını sağlar.



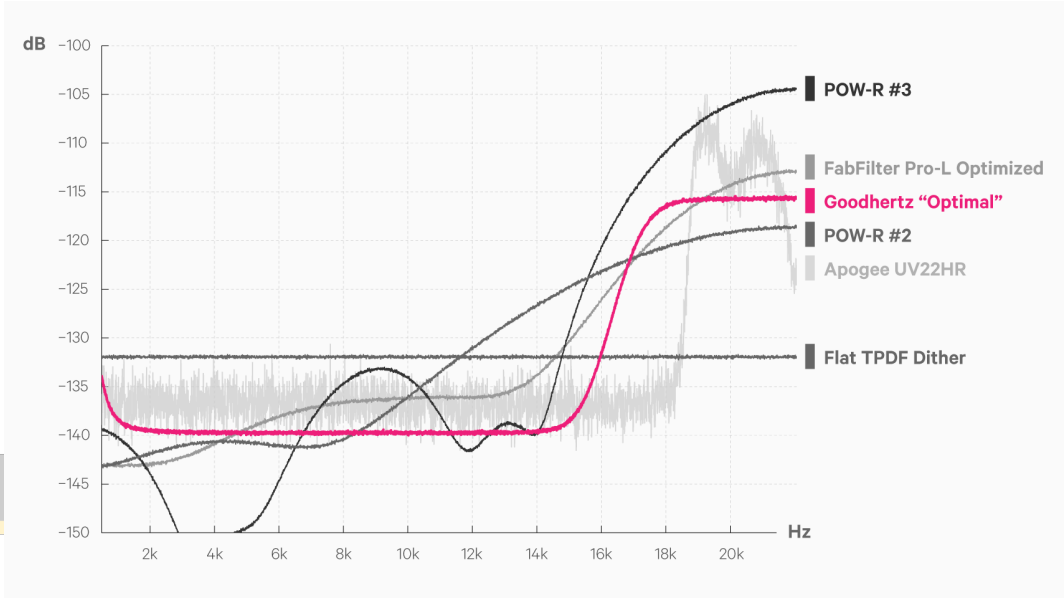
Dithering işlemi sonrasında ortaya çıkan gürültüyü, yine toplam gürültü enerjisini değiştirmeden[15], daha az duyulabilir hale getirme işlemine ise **gürültü şekillendirme (noise shaping)** adı verilir. Bu işlem gürültünün enerjisini insan kulağının daha az hassas olduğu bölgelerde arttırıp, daha hassas olduğu bölgelerde azaltmak olarak tarif edilebilir. Gürültüyü eşit gürlük eğrisi[16] kullanarak şekillendirmek gürültü şekillendirme yöntemlerinden biridir.

FMY-WEB.GITHUB.IO

[14] Ve belli bir süre sonra duyulmamaya başlar. İnsan beyni sabit gürültüyü belirli bir süre sonra filtreler. Hakkında düşünürseniz etrafınızda gürültü yayan bir sürü ses kaynağının olduğunu farkedebilirsiniz. Hakkında düşünmezseniz, hiçliğin ortasında düşen bir ağacın çıkarttığı ses gibi, var olmayacaklardır.

[15] Mümkün olsa toplam gürültü enerjisini değiştirmek isterdik tabiki. Ama maalesef mümkün değil. Yapabileceğimiz en iyi şey toplam gürültü enerjisini "arttırmadan" müdahalede bulunmak.

[16] Eşit gürlük eğrisini hatırlamıyorsanız logaritma ders notunun son bölümüne bakabilirsiniz.



Bu sayede algılanan dinamik alanı[17] da gerçek dinamik alandan daha büyükmüş gibi duyurmak mümkündür[18].[19]

MALİK ENES
ŞAFAK
PMY-WEB.GITHUB.IO

[17] Dinamik alan, sinyalin maksimum ulaşabileceği seviye ile sabit gürültünün olduğu seviye arasındaki alanı ifade eder. Bir sinyal dinamik alanın üzerine çıkarsa bozulur, dinamik alanın altında kalırsa gürültünün içerisinde kaybolur.

[18] Psikoakustik olarak.

[19] Bu bölümde anlatılan her şeyi mükemmel bir şekilde görsel olarak anlatan Akash Murthy'nin Digital Audio Fundamentals video serisine ulaşmak için https://www.youtube.com/playlist?list=PLbqhA-NKGP6B6V_AiS-jbvSzdd7nbwwCw

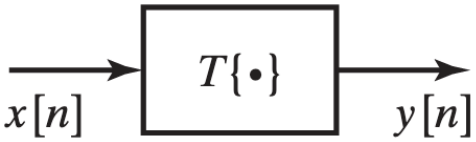
Ayrık Zamanlı Sistemler (Discrete Time Systems)

Sistem

Girişine gelen sinyali dönüştürerek bir çıkış üreten yapıya sistem adı verilir. Ses sinyalini işlemek için kullandığımız her şey bir sistemdir. Matematiksel olarak bu dönüşümü

$$y[n] = T\{x[n]\}$$

olarak ifade edebiliriz. Burada $x[n]$ giriş sinyalini, T (transformation, dönüşüm) sistemi ve $y[n]$ ise çıkış sinyalini gösterir. Bu ifadeyi aynı zamanda şematik olarak da aşağıdaki gibi gösterebiliriz.



Örneğin ideal bir **gecikme (delay)** sistemini aşağıdaki gibi ifade edebiliriz.

$$y[n] = x[n - n_d]$$

Buradaki ifadede n_d sistemin geciktirme miktarını belirler. Başka bir örnek olarak en basit ayrık zamanlı yüksek geçiren (high pass) ve alçak geçiren (low pass) filtresini aşağıdaki gibi tanımlayabiliriz.

$$y[n] = \frac{x[n] + x[n-1]}{2} \qquad (low\ pass)$$

$$y[n] = \frac{x[n] - x[n-1]}{2} \qquad (high\ pass)$$

Bu iki ifadede low pass filtre için giriş sinyalinin o anki örneği (n) ile bir önceki örneğinin $(n-1)$ toplamı[20], high pass filtre için giriş sinyalinin o anki örneğinden bir önceki örneğin çıkarılması[21] çıkış sinyalini vermektedir[22].

[20] https://www.desmos.com/calculator/lyawclmuhi

[21] https://www.desmos.com/calculator/6pyspmnx4o

[22] Neden bu işlemler sinyal üzerinde filtreleme yapıyor?

Lineer Sistemler (Linear Systems)

Sistemler içerisinde lineer sistemler **süperpozisyon (superposition)** presibiyle tanımlanır. Eğer $y_1[n]$ ve $y_2[n]$ sistemin $x_1[n]$ ve $x_2[n]$ girişine karşı verdiği sonuç olarak kabul edersek, bu sistemin lineer olduğunu söyleyebilmemiz için

$$T\{x_1[n] + x_2[n]\} = T\{x_1[n]\} + T\{x_2[n]\} = y_1[n] + y_2[n]$$

ve

$$T\{ax[n]\} = aT\{x[n]\} = ay[n]$$

özelliklerini karşılıyor olması gerekir. İkinci özellikte a sabit bir çarpandır. İlk özellik **birleşme özelliği (additivity property)**[23], ikinci özellik ise **ölçeklenme özelliği (scaling property) ya da homojenlik (homogeneity)**[24] olarak isimlendirilir. Bu iki özellik birlikte süperpozisyon prensibini oluşturur.

$$T\{ax_1[n] + bx_2[n]\} = aT\{x_1[n]\} + bT\{x_1[n]\}$$

Lineer olmayan bir sistem örneği vermemiz gerekirse

$$y[n] = \log_{10}(x[n])$$

eşitliğinde sistem, kendine gelen her giriş örneğinin logaritmasını alarak çıkış üretmektedir. Böyle bir sisteme tek bir örnek içeren $x_1 = [1]$ ve $x_2 = [10]$ sinyallerini verirsek

$$\log_{10}(1 + 10) \neq \log_{10}(1) + \log_{10}(10)$$

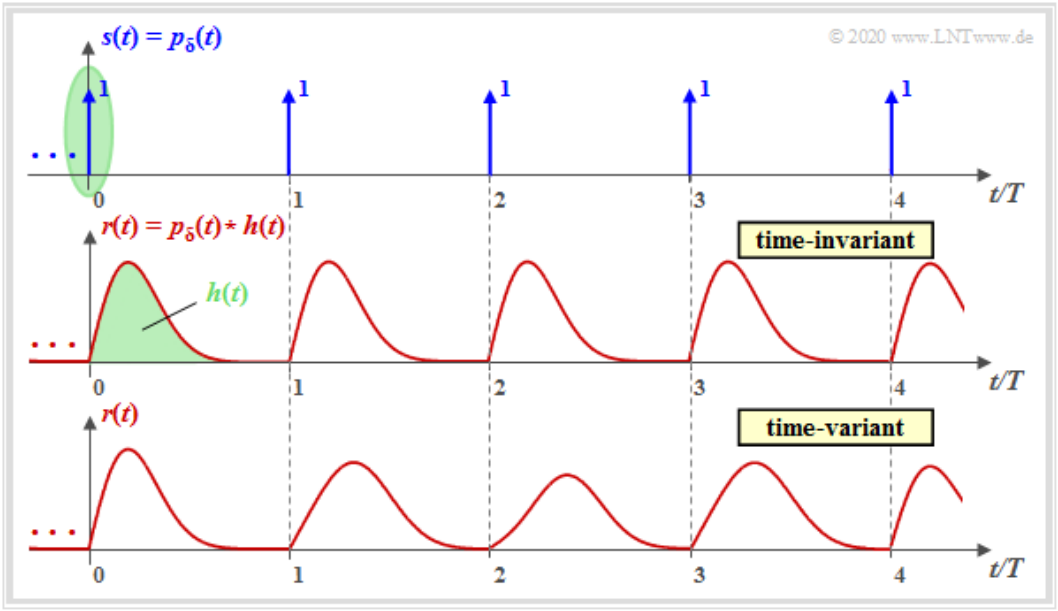
sistem birleşme özelliğini sağlamadığı için lineer değildir.

Zamanda Değişmez Sistemler (Time-Invariant System)

Sistemler içerisinde zamanda değişmez sistemler giriş sinyalinin zamanda geciktirilmesi durumunda çıkış sinyalinin de birebir aynı gecikmeye sahip olacağı sistemlerdir. Bir sistemin zamanda değişmez olduğunu kanıtlamak zor olsa da zamanda değişir olduğunu kanıtlamak için tek bir karşı örnek bulmak yeterlidir.

[23] Örneğin DAW içerisinde iki farklı kanalınız var ve bu kanalların ikisine de aynı şekilde filtre uygulamak istiyorsunuz. Kullandığınız filtrenin lineer bir sistem olduğunu kabul edersek (ki genellikle öyledir) birleşme özelliği sebebiyle iki kanala da aynı filtreyi uygulayıp mikslemeniz ile iki kanalın mikslenmiş halini filtreden geçirmeniz birebir aynı sonucu vermelidir.

[24] Örneğin DAW içerisinde bir kanalınız var ve bu kanala hem bir filtre uygulamak hem de sesini kısmak istiyorsunuz. Yine, kullandığınız filtrenin lineer bir sistem olduğunu kabul edersek kanalınızın önce sesini kısıp sonra filtreyi uygulamanız ile önce filtreyi uygulayıp daha sonra sesini kısmanız birebir aynı sonucu vermelidir.



Zamanda değişir bir sistem örneği vermemiz gerekirse

$$y[n] = x[Mn]$$

eşitliğinde sistem, n indisini M pozitif tamsayısıyla çarparak x sinyali içerisindeki örnekleri okumaktadır. Bu sistem (ideal olmayan bir şekilde) örnekleme frekansını azaltmak için kullanılabilir. x sinyali içerisindeki örneklerin

$$x[n] = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]$$

ve M sayısının 2 olduğunu kabul edersek

$$y_1[n] = x[Mn] = [0,2,4,6,8]$$

olacaktır. Fakat okuduğumuz sinyali 1 örnek geciktirirsek

$$y_2[n] = x[Mn - 1] = [0,1,3,5,7,9]$$

olacaktır[25]. Zamanda değişmez olan bir sistem için $y_1[n-1] = y_2[n]$ olması gerekirdi. Fakat bu iki sonucu karşılaştırdığımızda, ortaya çıkan sonuç birbirinden tamamen farklı olduğu için, aynı olmayacağı kesin. Bu sebeple böyle bir sistem zamanda değişmez değildir[26].

[25] Sinyali 1 örnek geciktirmemiz durumunda ilk okuyacağımız örnek (yani sıfır zamanında okuyacağımız örnek) $Mn - 1 = 2 * 0 - 1 = 0 - 1 = -1$ olacaktır. Sinyalimizin başlangıç ve bitişinin dışındaki bütün zamanlarda 0 olduğunu varsayarız. Bu sebeple -1. örneğimizin 0 olduğunu kabul ediyoruz.

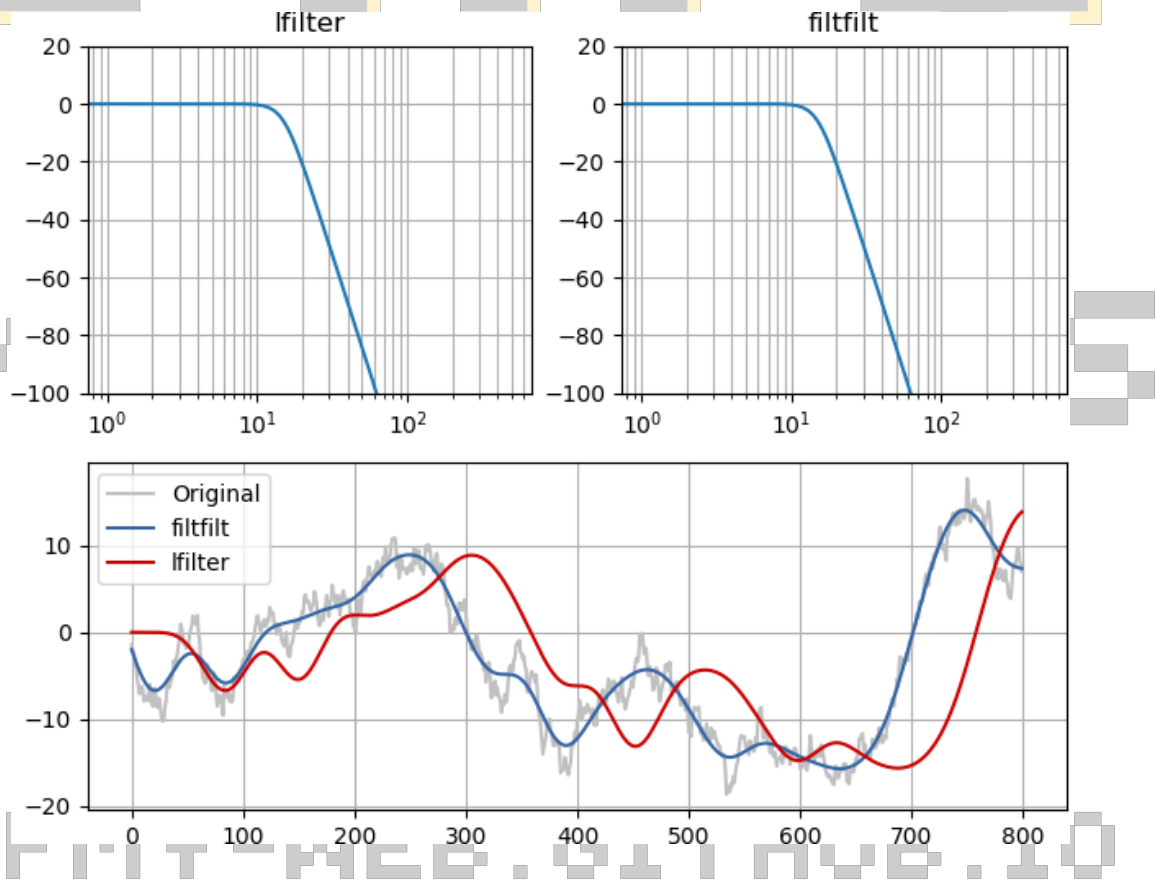
[26] Zamanda değişmezliği şöyle de düşünebilirsiniz; örneğin odanızda bir gitar kaydı alıyorsunuz. Kayıt tuşuna saat 13:30'da basmanızla 13:31'de basmanız arasında, eğer odanızda ciddi bir değişiklik yapmadıysanız, bir fark olmayacaktır. Böylelikle diyebiliriz ki odanız sinyal için zamanda değişmez bir sistemdir. Eğer kaydı 1 dakika sonra almaya başlamanız durumunda ses tamamen değişecek olsaydı bu durumda zamanda değişir bir sistem olurdu (ve bu mesleği yaparken hayatlarımız çok zorlaşırdı).

Nedensellik (Causality)

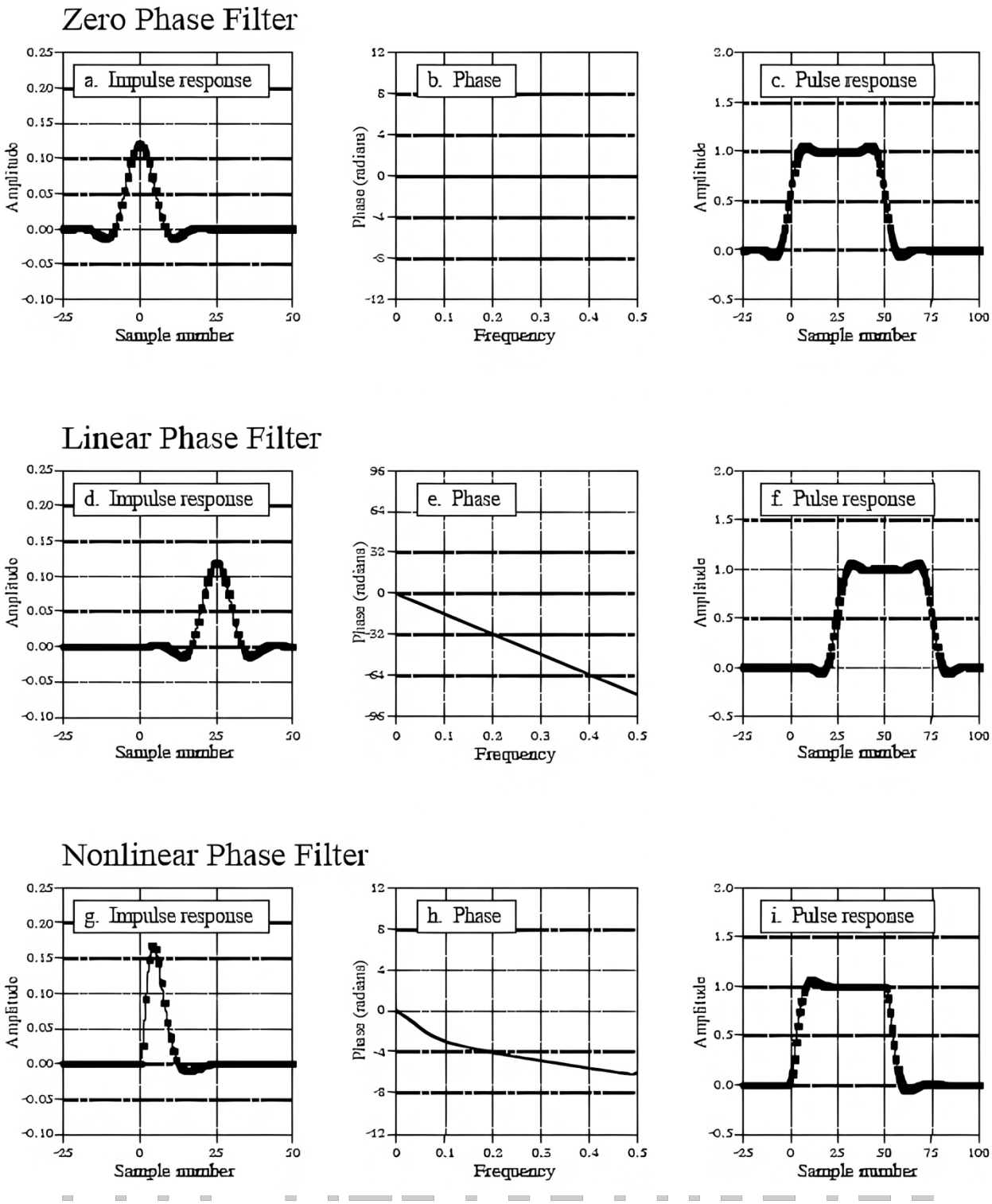
Bir sistem sadece o anki ve o andan önceki örneklere bağlı olarak bir sonuç veriyorsa bu sistem **nedenseldir (causal)**. Yani sistem sadece bir etki sonucu tepki üretir. Eğer sistem gelecekteki örnekleri kullanarak bir sonuç üretiyorsa bu sistem **nedensel olmayan (non-causal)** bir sistemdir. Yani sistem bir henüz bir etki olmadan da sonuç üretiyordur.

$$y[n] = x[n+1] - x[n]$$
$$y[n] = x[n] - x[n-1]$$

Yukarıdaki iki eşitlikte üstteki eşitlik gelecekteki örneğe bağlı olarak sonuç ürettiği için nedensel olmayan, alttaki eşitlik sadece şu anki ve geçmişteki örneğe bağlı olarak sonuç ürettiği için nedensel bir sistemdir.



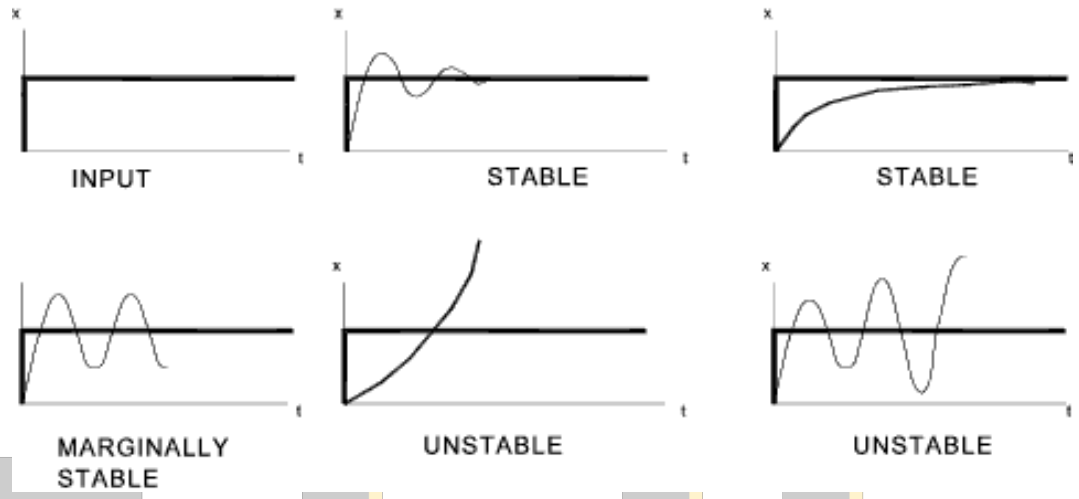
Yukarıdaki görselde lfilter nedensel bir filtre, filtfilt ise nedensel olmayan bir filtredir. Sıfır faz bir ses filtresi ancak nedensel olmayan bir filtre ile oluşturulabilir. Fakat gerçek dünyada nedensel olmayan bir filtre oluşturmak imkansızdır, çünkü geleceği önceden öngöremeyiz ya da henüz var olmayan bir veriyle işlem yapamayız. Bu sebeple sıfır faz bir filtre ancak dijital ortamda mümkün olabilir.



Stabilite (Stability)

Bir sistem bütün sınırlı giriş[27] sinyallerine karşılık sınırlı çıkış sinyali üretiyorsa bu sistemin stabil olduğunu gösterir.

[27] Sınır, sinyal işleme bağlamında, sinyalin genliğinin belirli bir alanın altında kalması anlamında kullanılır. Bir sinyal sınırlı değilse sonsuza kadar büyüyor anlamına gelir.



LTI (Linear time-invariant) Sistemler

Eğer bir sistem hem lineer hem de zamanda değişmez ise sinyal işleme alanında büyük bir öneme sahiptir. Bu tür sistemler **dürtü tepkisi (impulse response)** ile kolaylıkla karakterize edilebilir ve sistemin sinyal üzerinde yaptığı işlem bilinmese bile taklit edilebilir. Eğer bir sistemin dürtüye karşı verdiği tepki $h[n]$ ise

$$y[n] = \sum_{k=-\infty}^{+\infty} x[n]h[n-k]$$

işlemi ile kolaylıkla $x[n]$ sinyaline uygulanabilir. Bu işleme **konvolüsyon (convolution) ya da evrişim** adı verilir ve genellikle $*$ işareti ile gösterilir.

$$y[n] = x[n] * h[n]$$

Lineer ve zamanda değişmez bir sistemin dürtü tepkisi sistemin bütün özelliklerini kapsar.

FMY-WEB.GITHUB.IO